

2017 —2018 学年第二 学期期末考试试题

课程名称: 数据结构 (A) 出题教师: 桂葆, 李向前, 孙永奇, 黄冠千

专业: _____ 班级: _____ 姓名: _____ 学号: _____

题号	一	二	三	四	五	六	七	总分
得分								
阅卷人								

备注: 请把答案写在答题卡上。

一、选择题(每题 2 分, 共 20 分)

1. 下面程序段的时间复杂度是 ()。

```
for (i = 1; i <= n; i++)
{
    x = 0;
    for (j = 1; j <= i; j++) { x = x + A[i] * B[j] * x; }
}
```

A. $O(n)$ B. $O(n^2 + 2n)$ C. $O(\log n)$ D. $O(n^2)$

2. 栈和队列的共同点是 ()。

A. 都是先进后出 B. 都是后进先出
C. 只允许在端点处插入和删除元素 D. 没有共同点

3. 已知主串 $S = 'abababababab'$, 模式串 $T = 'ababab'$, 则 next 函数值及 nextval 函数值为 ()。

A. 011222212 和 010222201 B. 011222234 和 010____01
C. 011222212 和 010222104 D. 011222234 和 010_22104

4. 已知广义表 $L = ((x, y), z, (u, t, v))$, 从 L 表中取出原子项 v 的运算是 ()。

A. $\text{head}(\text{tail}(\text{tail}(L)))$ B. $\text{head}(\text{head}(\text{tail}(\text{tail}(L))))$
C. $\text{head}(\text{tail}(\text{head}(\text{tail}(L))))$ D. $\text{head}(\text{tail}(\text{head}(\text{tail}(\text{tail}(L)))))$

5. 一个具有 2014 个结点的二叉树的高度为 ()。

A. 11 B. 10 C. 11 到 2014 之间 D. 10 到 2014 之间

6、在遍历中序线索二叉树时，结点的后继是其（ ）。

- A. 左子树中最右下的结点 B. 前驱线索指向的结点
C. 左子树中最左下的结点 D. 不能确定

7、用邻接矩阵来表示有 5 个顶点的无向图，该图中有 6 条边，则邻接矩阵中有（ ）个非零元。

- A. 5 B. 6 C. 10 D. 12

8、下列关于最小生成树的说法中，正确的是（ ）。

(1) 最小生成树的代价唯一

(2) 权值最小的边一定会出现在所有的最小生成树中

~~(3) 用 Prim 算法从不同顶点开始得到的最小生成树的形态一定相同~~

~~(4) Prim 算法和 Kruskal 算法得到的最小生成树的形态一定不相同~~

- A. 仅 (1) B. 仅 (2) C. 仅 (1) (2) D. (1) (2) (4)

9、一棵 3 阶 B 树中，有 10 个关键字

则其叶子结点数为（ ）。

- A. 9 B. 10 C. 11 D. 12

10、若需要在 $O(n \log n)$ 的时间内完成对数据的排序，且要求占用辅助空间最少，则可选的排序方法是（ ）。

- A. 快速排序 B. 归并排序 C. 堆排序 D. 直接插入排序

二、填空题(每题 1 分，共 10 分)

1、一种数据结构的抽象数据类型 (ADT) 表示包括三部分：_____、_____、_____。

2、在有 n 个数据元素的顺序查找表中，等概率情况下顺序查找的平均查找长度为_____。

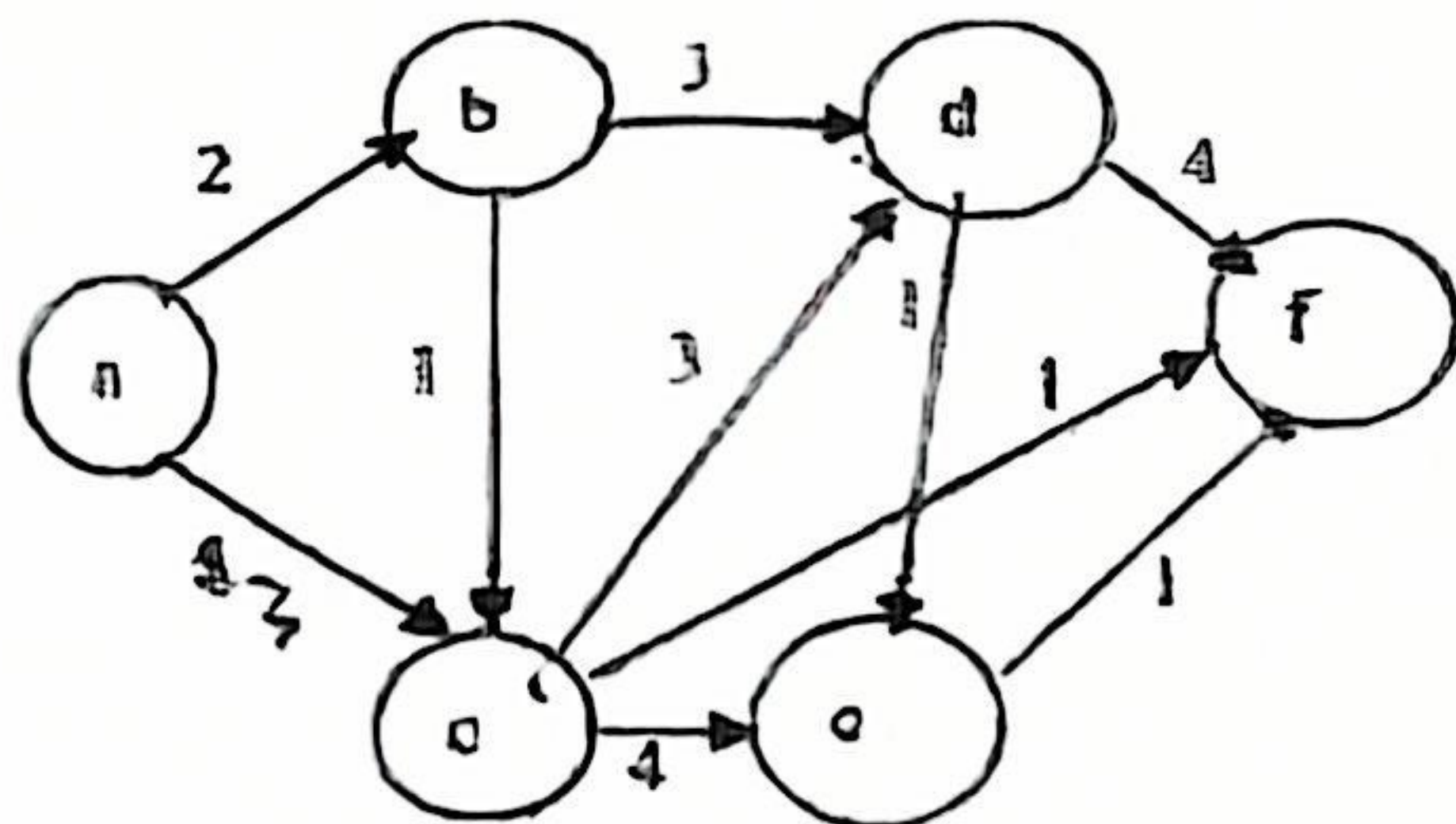
3、循环队列存储在数组 $A[0..6]$ 中，其头、尾指针分别是 $front=3$ 和 $rear=0$ ，则队列中的元素个数为_____。

4、一个 n 阶对称矩阵 $A[1..n, 1..n]$ ，将其下三角部分(包括主对角线上的元素)以行序为主序存放在一维数组 $S[1, n(n+1)/2]$ 中，则矩阵元 $A_{5,4}$ 在 S 中的位置是_____。

5、在只有度为 0 和度为 k 的结点的 k 叉树中，设度为 0 的结点有 n_0 个，度为 k 的结点有 n_k 个，则 n_0 与 n_k 的关系是_____。

6、如果有向图中有 n 个顶点和 e 条弧，那么它的邻接表中有 $2e$ 个弧结点。

7、如下有向带权图，若采用迪杰斯特拉 (Dijkstra) 算法求源点 a 到其它各顶点的最短路径，得到的第一条最短路径的目标顶点是 b ，第二条最短路径的目标顶点是 c ，后续得到的其余各最短路径的目标顶点依次是 f, d, e



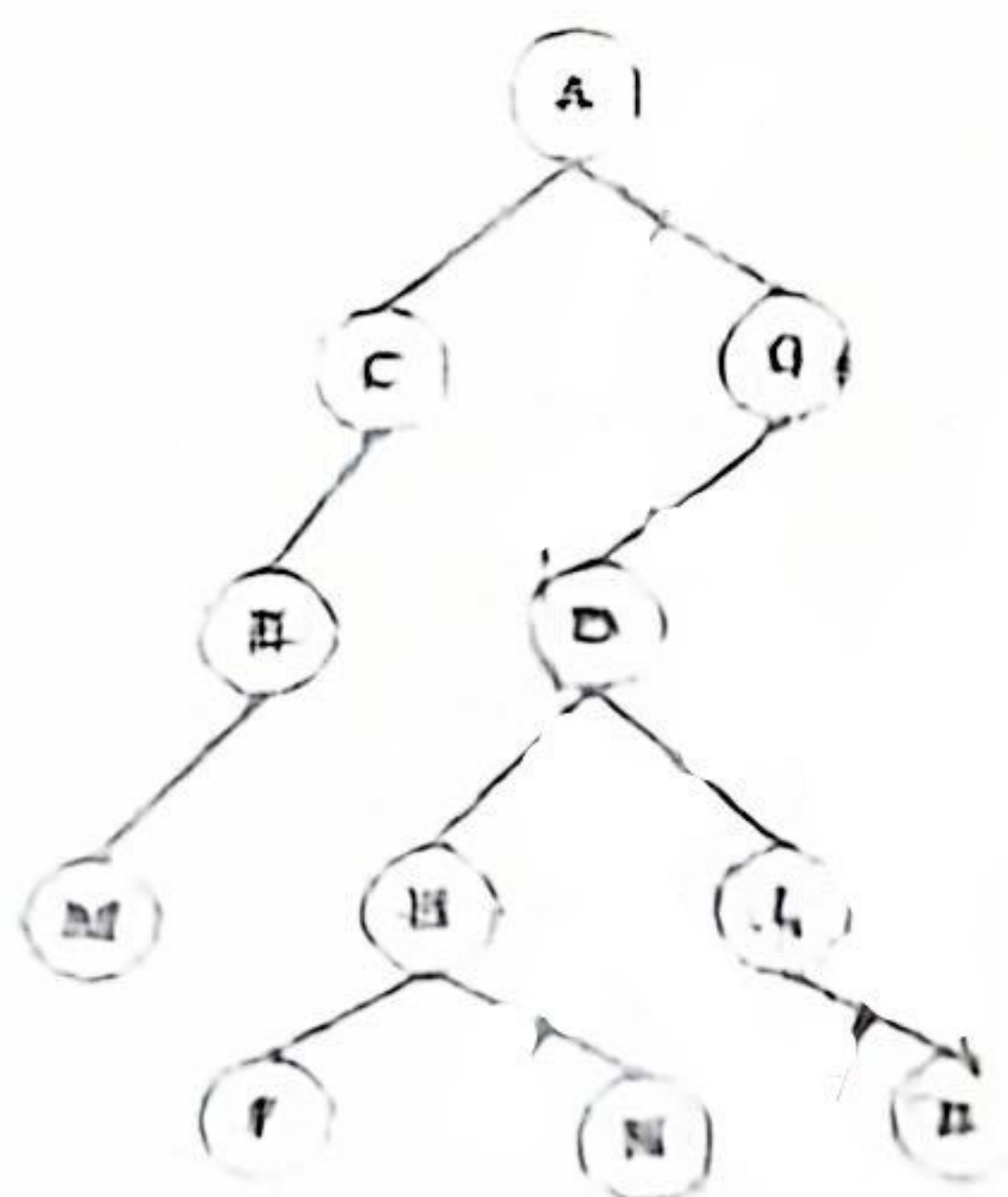
8、平衡二叉树上所有结点的平衡因子只可能是 $-1, 0, 1$

9、从有序表 (10、19、32、43、56、78、85、95) 中折半查找元素 85 时，查找成功时的比较次数为 4

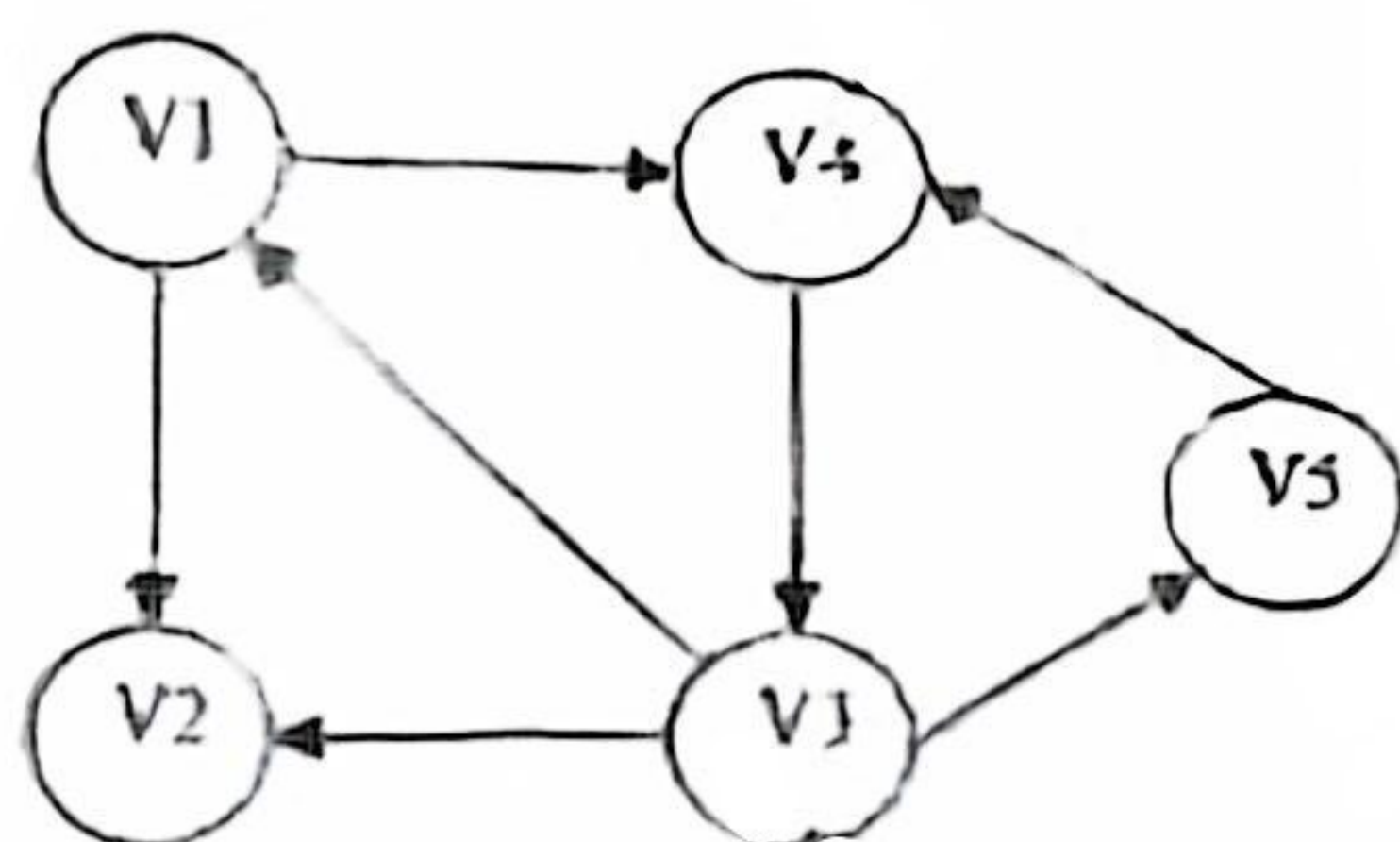
10、对关键字序列 (742, 89, 53, 134, 9, 91) 采用链式基数排序方法由大到小降序排序，第一趟排序后的结果为 742, 134, 91, 89, 53, 9

三、作图题 (共 3 小题，共 15 分)

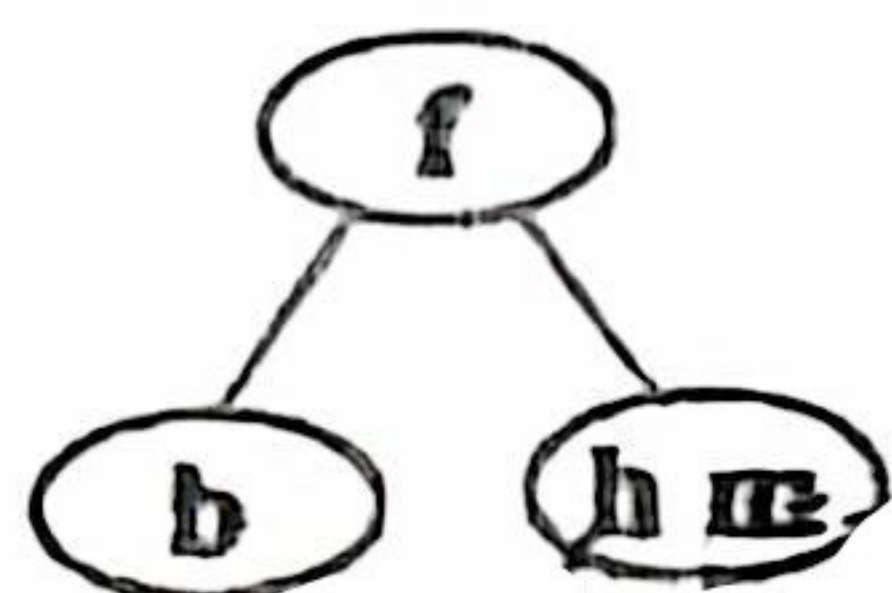
1、把用孩子兄弟链表表示的二叉树转换成相应的森林。(5 分)



2、画出如下所示有向图的逆邻接链表，并求出每个顶点的度。(5 分)



3. 已知一颗 2-3 树 (3 阶 B-树) 如图所示, 画出在这颗树上依次插入 d, p, l, s 后的状态图。 (5 分)



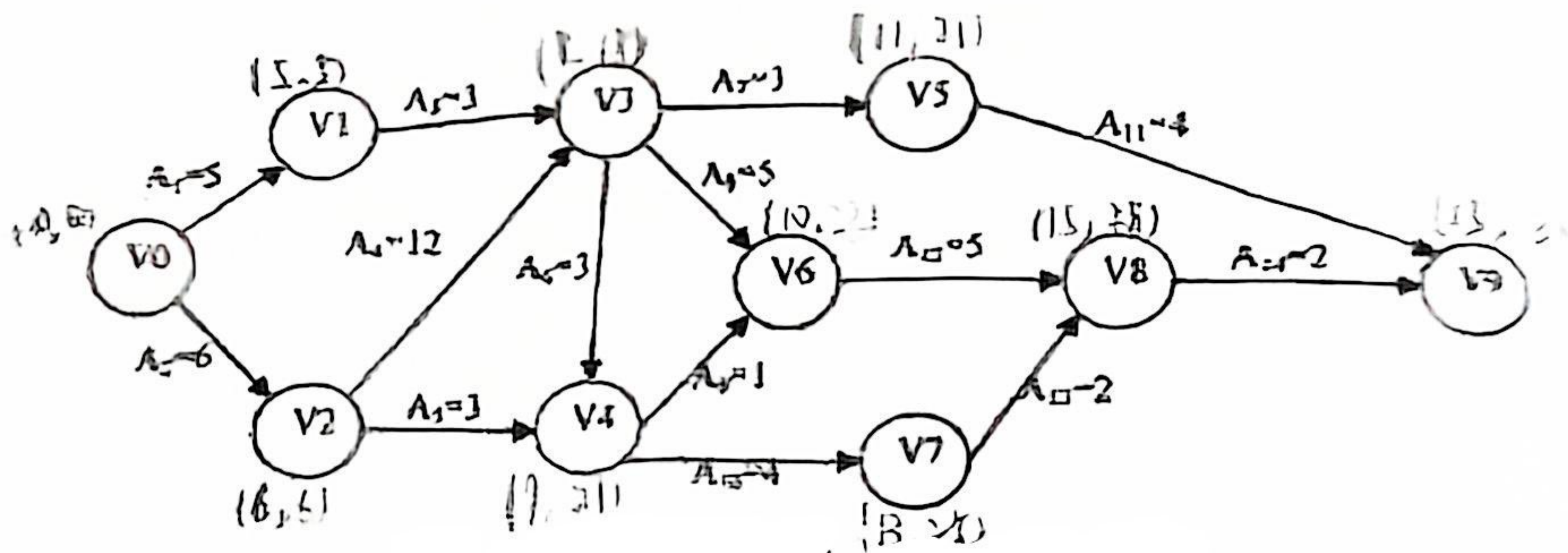
四、解答题 (共 3 小题, 共 25 分)

1. 已知有 8 个叶子结点的哈夫曼树的存储结构如下图所示, 画出哈夫曼树并对这 8 个叶子结点进行前缀编码。 (8 分)

id	weight	parent	lchild	rchild
1	3	9	0	0
2	25	14	0	0
3	7	10	0	0
4	8	10	0	0
5	14	12	0	0
6	23	18	0	0
7	3	9	0	0
8	11	11	0	0
9	8	11	1	7
10	15	12	3	4
11	18	13	5	9
12	29	14	6	10
13	42	15	8	11
14	58	15	2	12
15	100	0	13	14

2. 假设一个工程的进度计划用下图所示 AOE 网表示, (8 分)

- (1) 求出每个事件的最早发生时间和最晚发生时间;
- (2) 求出该工程完工至少需要多少时间;
- (3) 求出所有关键路径和关键活动。



3、已知一组关键字(19, 14, 23, 1, 68, 20, 84, 27, 55, 11, 10, 79)，哈希函数为： $H(key) = key \text{ MOD } 13$ ，用链地址法处理冲突，画出散列表存储状态图，并求出等概率情况下查找成功时的平均查找长度。(9分)

五、算法阅读题 (共 10 分)

阅读下列算法并回答问题。

```
#define MAX_VERTEX_NUM 20

typedef struct ArcNode {
    int adjvex;
    int weight;
    struct ArcNode *nextarc;
} ArcNode;

typedef struct VNode {
    char data;
    ArcNode *firstarc;
} VNode, AdjList[MAX_VERTEX_NUM];

typedef struct {
    AdjList vertices;
    int vexnum, arcnum;
} ALGraph;

int LocateVex(ALGraph G, char c) {
    int i;
    for (i = 0; i < G.vexnum; i++) {
        if (G.vertices[i].data == c) return i;
    }
    return -1;
}
```



```

void InitGraph(ALGraph &G)
{
    int v1, v2, i;
    ArcNode *p, *q;
    char b[7] = {'A', 'B', 'C', 'D', 'E', 'F', 'G'};
    int u[11][3] = {{0, 1, 7}, {0, 3, 5}, {1, 2, 8}, {1, 3, 9}, {1, 4, 7}, {2, 4, 5},
                    {3, 4, 15}, {3, 5, 6}, {4, 5, 8}, {4, 6, 9}, {5, 6, 11}};
    G.vexnum = 7; G.arcnum = 11;
    for (i = 0; i < G.vexnum; i++) {
        G.vertices[i].data = b[i];
        G.vertices[i].firstarc = NULL;
    }
    for (i = 0; i < G.arcnum; i++) {
        v1 = u[i][0];
        v2 = u[i][1];
        p = (ArcNode *)malloc(sizeof(ArcNode));
        p->weight = u[i][2];
        p->adjvex = v2;
        p->nextarc = NULL;
        if (!G.vertices[v1].firstarc)
            G.vertices[v1].firstarc = p;
        else {
            q = G.vertices[v1].firstarc;
            while (q->nextarc)
                q = q->nextarc;
            q->nextarc = p;
        }
        p = (ArcNode *)malloc(sizeof(ArcNode));
        p->weight = u[i][2];
        p->adjvex = v1;
        p->nextarc = NULL;
        if (!G.vertices[v2].firstarc)
            G.vertices[v2].firstarc = p;
        else {
            q = G.vertices[v2].firstarc;
            while (q->nextarc)
                q = q->nextarc;
            q->nextarc = p;
        }
    }
}

```



```

bool add[MAX_VERTEX_NUM];
int adjacent[MAX_VERTEX_NUM] = { 0 };
int sumweight = 0;
int i, j;
for (i = 0; i < G.vexnum; i++)
{ lowcost[i] = 100000;
  add[i] = false;
}
add[0] = true; adjacent[0] = 0;
ArcNode* p = G.vertices[0].firstarc;
while (p != NULL)
{ lowcost[p->adjvex] = p->weight;
  p = p->nextarc;
}
for (i = 0; i < G.vexnum; i++)
{
  int min = 10000;
  int v = -1;
  for (j = 1; j < G.vexnum; j++)
  { if (!add[j] && lowcost[j] < min)
    { min = lowcost[j];
      v = j;
    }
  }
  if (v != -1)
  { printf("%6s %6s %6d\n", G.vertices[adjacent[v]].data,
    G.vertices[v].data, lowcost[v]); // 输出运行结果
    add[v] = true;
    sumweight += lowcost[v];
    p = G.vertices[v].firstarc;
    while (p != NULL)
    { if (!add[p->adjvex] && p->weight < lowcost[p->adjvex])
      { lowcost[p->adjvex] = p->weight;
        adjacent[p->adjvex] = v;
      }
      p = p->nextarc;
    }
  }
}
}

void main() {
  ALGraph pg;
  InitGraph(pg);
  func(pg);
}

```

(1) 说明程序的功能：(2 分)

(2) 画出图: (2分)

(3) 写出程序的运行结果: (6分)

六、算法填空题。(每空2分,共10分)

下列算法实现记录的关键词的升序排序,请把算法补齐。

```
typedef int KeyType;
```

```
typedef struct {
    KeyType key;
} RecType;
```

```
typedef struct {
    RecType r[101];
    int length;
} SqList;
```

```
void SI(SqList *pL, int dk) {
    int i, j, m;
    m = pL->length;
    for (i = dk + 1; i <= m; ++i)
        if (____ (1) ____ )
            { pL->r[0] = pL->r[i];
              for (____ (2) ____; j > 0 && (pL->r[0].key < pL->r[j].key); ____ (3) ____ )
                  pL->r[j + dk] = pL->r[j];
              ____ (4) ____ = pL->r[0];
            }
}
```

```
}
void SSort(SqList *pL)
{
    int k, L, dlim[5];
    L = 4;
    dlim[0] = 5; dlim[1] = 3; dlim[2] = 2; dlim[3] = 1;
    for (k = 0; k < L; ++k)
        SI(pL, ____ (5) ____ k);
}
```

```
void main()
{
    int i, n = 10;
    SqList *pL = malloc((n+1)*sizeof(RecType));
    int a[10] = {49, 38, 65, 97, 76, 13, 27, 49, 55, 4};
    for (i = 0; i < n; ++i) {
        pL->r[i + 1].key = a[i];
    }
    pL->length = n;
}
```


}

七、算法设计题（共 10 分）

对于一个线性链表，请给出计算结点个数的函数实现。包括递归和非递归两种实现方法。

已知链表结点的定义如下：

```
typedef struct node {  
    ElemType data; // 数据域  
    struct node *next; // 指针域  
} node;
```